

Independent Project - A Flink-Based Project on TPC-H Query Q3

By Tuo Xiong - 21186900

<https://github.com/TakuQAQ/A-Flink-Based-Project-on-TPC-H-Query-Q3>

1. Background and Related Work

1.1 Project Background

The TPC-H benchmark is a widely accepted industry standard for evaluating the performance of decision support systems and complex analytical queries. Among these, TPC-H Query 3 (Q3), also known as the Shipping Priority Query, is a typical heavily-loaded query. It retrieves the top 10 unshipped orders with the highest revenue within a specific shipping window, filtering across three large tables: Customer, Orders, and Lineitem.

Traditionally, TPC-H queries are executed using batch processing or standard SQL engines. However, as modern businesses demand real-time insights, migrating TPC-H Q3 into a streaming environment using Apache Flink has become a highly relevant and valuable scenario. This project implements a real-time, low-latency version of TPC-H Q3, processing simulated historical event streams to output the rolling dynamic Top-10 revenue ranking.

1.2 Related Work and Technical Challenges

In a standard SQL or batch system, Q3 relies heavily on multi-table JOIN operations followed by a global ORDER BY ... LIMIT 10. Translating this logic into a distributed stream processing architecture with Apache Flink introduces two major technical bottlenecks:

- ❖ *Processing Pre-Shuffled Distributed Datasets:* To benchmark true distributed performance, the raw TPC-H data used in this project is pre-shuffled across multiple partition files rather than read from a single sequential source. Managing state and correctly aligning events (such as ensuring an Order event is recognized before its corresponding Lineitem details arrive) across decentralized partitions requires robust, bucketed state management within Flink's KeyedProcessFunction.
- ❖ *Rate-Controlled Streaming for Realistic Simulation:* Reading static CSV files sequentially at maximum disk speed does not accurately reflect a live production environment. To simulate actual business operations, the pipeline requires an explicit ingestion rate-control (throttling) mechanism. Controlling the processing speed allows the system to properly generate and advance time semantics (such as Processing Time and Watermarks).

To address these conditions, this project implements a specialized hybrid architecture. By utilizing an offline pre-processing utility to structurally combine data fields and utilizing a rate-controlled source, the pipeline successfully replicates a live enterprise data feed, allowing Apache Flink to perform stable, highly scalable distributed Top-N aggregation.

2. Description of the Query and the Algorithm

2.1 Official Query Details and Formula

The TPC-H Q3 (Shipping Priority Query) retrieves the shipping priority and potential revenue of the top 10 unshipped orders with the largest revenue. Officially, the revenue is calculated using the following mathematical formula applied to each matching line item:

$$\text{Revenue} = l_extendedprice * (1 - l_discount)$$

The query performs a three-way join across Customer, Orders, and Lineitem based on the following relational criteria: $c_custkey = o_custkey$, $l_orderkey = o_orderkey$, and filters where customer's market segment must match a specific target (e.g. BUILDING), the order date must be before a given date (T), and the ship date must be after that same date (T) to guarantee the order remains unshipped during the target window. The final output groups the revenue by $o_orderkey$, $o_orderdate$, and $o_shippriority$, sorting the results in descending order of total revenue, and limits the output to the top 10 rows.

2.2 Rolling Streaming Top-10 Algorithm

In a real-time streaming architecture, instead of a one-time static calculation, the system must continuously ingest a live stream of data and provide a rolling, dynamic Top-10 view. To achieve high throughput and avoid centralized bottlenecks, a Two-Stage Distributed Top-10 Algorithm was designed:

- ❖ *Order Creation / Accumulation:* When a new Order event enters the stream, the algorithm extracts its revenue (calculated via the official formula) and adds this value to the order's current state in the MapState.
- ❖ *Shipping / Revenue Deduction:* To faithfully simulate the "unshipped" requirement of Q3, when a shipping event for a specific line item enters the stream, the algorithm identifies the corresponding orderKey and deducts the shipped amount from its total. If an order is fully shipped, it is removed from the active tracking state.
- ❖ *Continuous Evaluation:* A processing-time or event-time timer triggers periodically (e.g. every few seconds) to scan the current active orders in the MapState, sort them by their real-time revenue, and output the latest rolling Top-10 ranking.

3. Implementation Detail

3.1 Flink SQL and DataStream Streaming API (Batch Mode)

The corresponding code: [Query03_SQL](#), [Query03DS_Batch](#)

This baseline approach utilizes the declarative Flink SQL API or the core DataStream API

running in BATCH execution mode. The three tables are registered as static catalogs. The program executes the literal TPC-H Q3 SQL text, utilizing standard relational join strategies (such as Hash Joins) and an internal global sort to output a single, definitive Top-10 result for a fixed date condition. This serves as our analytical benchmark for data correctness.

3.1.1 MyEvent

Utilization of the low-level DataStream API explicitly introduces custom object definitions (e.g. custom Order and Lineitem POJOs) to enforce strict type safety and optimize serialization performance during stream partitioning and joining.

3.1.2 BatchTrigger

Because it operates as a batch simulation within a streaming execution mode, a global window is used to accumulate the final results. To ensure the final sorting and Top-10 pruning occur only after the entire dataset has been ingested, a custom BatchTrigger is attached to the global window. The trigger registers an event-time timer at Long.MAX_VALUE. When the file reader finishes and the stream reaches its end, Flink automatically advances the watermark to infinity (Long.MAX_VALUE). This action fires the trigger, executing the global sorting and outputting a single, definitive batch-like Top-10 result.

3.2 Streaming API with Separately Pre-Sorted Streams (One Stream)

The corresponding code: [Query03DS_Streaming](#)

3.2.1 MySort

To pivot from static batch execution to a real-time rolling streaming system, this approach processes live, continuous data feeds. To optimize the processing flow and provide a structured timeline for Flink's time semantics, an offline sorting phase is introduced using a custom utility called MySort. The MySort utility parses the data using a pipe (|) delimiter, dynamically extracts target fields via specialized indices (such as the date field at index 4 for Orders), and sorts the records in ascending order before writing them out to individual files (sorted_orders.csv and sorted_lineitem.csv).

3.2.2 DateParser

During runtime, these pre-sorted tables are fed into Flink as dual streaming sources. To drive the periodic evaluation, event-time semantics are initialized. Because MySort guarantees that timestamps arrive in strict chronological order, the pipeline assigns a monotonically increasing watermark strategy. As each event passes through the source, its date string is parsed via DateParser.parseDate into a millisecond timestamp, allowing Flink's internal watermarks to advance smoothly and reliably behind the incoming records.

3.2.3 MapLoader

To bypass the computational overhead of joining massive historical side-tables inside Flink's

runtime state backend, a static pre-aggregation and look-up strategy was designed. Implemented via the MapLoader utility, the system scans the raw static files prior to running the Flink pipeline, constructing memory-efficient lookup tables (HashMap) to enrich incoming data streams on the fly. The MapLoader handles two main static extractions:

- ❖ *Market Segment Evaluation:* The loadCustomerStatic method maps each unique custKey to its corresponding market segment (mktSegment), allowing the stream to quickly discard events that do not fall under the targeted Q3 query filters.
- ❖ *Order Revenue Aggregation:* The loadLineitemStatic method scans the global lineitem.csv file, isolates each orderKey, applies the standard TPC-H revenue calculation formula ($\text{extendedPrice} * (1 - \text{discount})$), and leverages Java's Map.merge function to pre-aggregate the total potential revenue for every individual order.

From an architectural perspective, the pre-aggregated maps generated by MapLoader function effectively as dimension tables rather than dynamic event streams. In enterprise production environments, streaming pipelines are typically reserved for high-velocity, transactional event streams (such as user clicks or live line-item updates). Static attributes—such as a customer's demographic profile (Customer) or the fixed reference values bound to an invoice upon its creation—are stored in relational data warehouses as dimension tables. By loading the customer profiles and pre-aggregated order revenues into memory beforehand, this project emulates a standard stream-to-dimension join pattern.

3.2.4 Simulator

To prevent Flink from consuming the pre-sorted CSV data instantly at maximum disk speed, a custom throttling layer was implemented using a custom ProcessFunction. This operator throttles the ingestion speed by mapping the historical TPC-H timeline to real-world execution time based on a tunable acceleration factor (speedUp). The core mechanism operates as follows:

- ❖ *Timeline Anchoring:* Upon initialization in the open() method, the function captures the actual system wall-clock startup time (startRealTime) and anchors it against the first logical date in the dataset, 1992-01-01 (globalFirstEventTs).
- ❖ *Dynamic Pacing:* For each incoming record, the function calculates the logical time elapsed since the baseline date (businessDelay). It divides this delay by the speedUp factor to determine exactly when the record should appear in real-world time. If that target time is in the future (sleepTime > 0), the thread is forced to sleep, effectively pacing the data stream to simulate a live, real-time enterprise data feed.

3.2.5 Processors

To handle the independent streams of Orders and Lineitems in a continuous real-time environment, the pipeline leverages an event-driven ingestion strategy. Rather than executing

a heavy relational join, both inputs are transformed into a standardized, unified data model (MyEvent.BusinessEvent) and merged into a single processing line (union).

The two streams use opposing financial logic to accurately maintain the active unshipped balances in the downstream unshippedState:

- ❖ *Order Stream (Revenue Insertion)*: When an Order event is instantiated, it pulls the pre-aggregated total order revenue from the static lineitemMap. When this event (isOrder = true) hits the stateful processor, it adds the positive total revenue to the unshippedState, initializing or incrementing the tracked backlog for that orderKey.
- ❖ *Lineitem Stream (Revenue Deduction)*: Conversely, as actual products ship out, they generate separate Lineitem events. These are mapped with a negative revenue value ($-\text{extendedPrice} * (1 - \text{discount})$) and marked as isOrder = false. When processed, this negative value is added to the state, which effectively deducts the shipped portion from the order's total active revenue.

To compute the rolling Top-10 ranking dynamically on the unified stream, a core stateful operator named timerTop10Processor was implemented via Flink's KeyedProcessFunction. This operator partitions the data stream using a bucketing key and relies on processing-time timers to trigger local updates every 5 seconds (interval = 5000). The implementation logic addresses two critical challenges in streaming analytics: tracking temporal state mutations and enforcing strict retrospective business rules.

Because streaming files are consumed rapidly, a single processing-time execution window may ingest events spanning multiple business dates. If Flink only tracked the absolute latest cumulative revenue, an upcoming transaction from a "future" business date would immediately contaminate the current evaluation. To isolate states across time boundaries, the unshippedState is declared as a MapState mapping an orderKey to a Tuple3. Each slot in this nested structure stores a Tuple2<Double, Long>, binding a specific historical revenue total to its exact logical business timestamp (ts). Here f0 represents the most up-to-date active version. If an arriving event matches this timestamp, its revenue is added. If a new business date arrives, the state cascades downward. Similarly, f1 and f2 act as sliding historical snapshots. When a new day triggers a state change, the records shift (f2 = f1, f1 = f0), rolling over the previous totals. This mechanism effectively retains up to three generations of historical data within the operator.

To strictly fulfill the TPC-H Q3 criteria—where evaluated order dates must be less than or equal to the active target simulated date—the sorting logic links the evaluation to Flink's internal Watermark. During the periodic refreshTop10 loop, the processor fetches the current watermark (threshold), which tracks the safe, fully processed simulated business time. The algorithm then iterates through the multi-version histories. It looks at f0, f1, and f2 sequentially, extracting the most recent value whose bound timestamp is less than or equal to

the watermark threshold (`historyState.fX.fl <= threshold`). Values tied to future dates are completely ignored for the current ranking.

This approach guarantees that even if a background thread has already buffered or calculated subsequent order revenues into the state backend, the localized sorting algorithm filters out those future entries. It ensures the console output reflects a true, chronological snapshot matching the simulated progression of business time.

3.3 Streaming API with Separately Pre-Sorted Streams (Parallel Streams)

The corresponding code: *Query03DS_Streaming_Parallel*

While the developed streaming pipeline runs seamlessly at smaller data scales (e.g. $SF = 1$), running the single-threaded (Parallelism = 1) approach under TPC-H $SF = 10$ pushes the system beyond the physical limits of a single CPU core. At $SF = 10$, the combined event stream scales up to approximately 75 million transactional records (15 million orders and 60 million line items). Under this volume, the 5-second periodic global refresh interval (interval = 5000) introduces a fatal architectural bottleneck for a single thread. In a single-parallelism architecture, the entire data stream—including data parsing, state updates, chronological filtering, and sorting—is funneled into a single task slot on a single worker node.

In this approach, a Two-Stage Distributed Top-10 Architecture was developed, splitting the computation into a highly optimized Local-to-Global pipeline. In the first stage, the unified event stream is shuffled across N parallel subtasks using a custom bucketing key (`orderKey % N`). Each independent subtask manages its own isolated partition of the `unshippedState`. When the periodic 5-second timer fires, each parallel slot only has to scan, filter, and sort its localized subset of data. Each slot then emits exactly its top 10 local orders downstream. In the second stage, a single downstream operator collects the outputs from all parallel branches. For instance, with a parallelism of 8, the global operator only needs to ingest and sort a maximum 80 records per reporting interval. It merges these pre-filtered lists, executes a definitive final sort, and outputs the global Top-10 ranking.

3.4 Streaming API with Combined and Pre-Sorted Unified Streams

The corresponding code: *Query03DS_Streaming_PreSorted*

In a dual-source pipeline, the Orders and Lineitem data streams progress at different speeds due to varied file reader speeds, network interleaving, or thread scheduling. This frequently causes a child Lineitem event to arrive at the Flink operator before its parent Orders event has been ingested, resulting in extensive state-waiting latency, bloated memory footprints, and potential data loss if state-clearing windows expire early.

To systematically eliminate this synchronization bottleneck, this approach introduces an offline external unification and pre-sorting phase prior to Flink ingestion. It blends all

transactions into a single, structurally unified schema (MyEvent.MergedBusinessEvent). The resulting combined file is sorted strictly based on ascending date and then descending isOrder. When a Lineitem record appears in the stream, its parent order is already securely instantiated inside the localized MapState. This completely bypasses the need for complex temporal stream joins or long-term caching buffers within Flink. The streaming pipeline operates with near-zero state overhead, eliminating out-of-order data friction and producing a stable, real-time "screen-flooding" Top-10 visualization on the console.

4. Experimental Setup and Result

4.1 Experimental Setup and Data Sets Used

To evaluate the scalability and throughput of the streaming pipeline, benchmarks were conducted using standard TPC-H relational datasets generated at two distinct scales: Scale Factor 1 (SF = 1, approx. 1GB) for baseline validation and Scale Factor 10 (SF = 10, approx. 10GB) for high-volume stress testing. The analytical workloads targeted three core tables: Customer, Orders, and Lineitem. The runtime environment was hosted on an Intel Core i9 high-frequency multi-core processor running Apache Flink in Standalone mode, matching Task Slot counts exactly to logical CPU threads to optimize parallel execution without thread contention.

4.1 Experimental Results and Discussion

The initial benchmarking evaluated the static processing methods under batch semantics (SF = 1, Parallelism = 4). When executing the analytical query on static files, the Flink SQL implementation achieved a total runtime of approximately 9 seconds. In contrast, the Streaming API configured in Batch Mode outperformed the SQL engine, completing the full dataset aggregation and ranking lifecycle in roughly 5 seconds. This performance divergence highlights the architectural efficiency of the custom Streaming API pipeline. In this scenario, the handcrafted Streaming API reduces execution latency by nearly 45%.

l_orderkey	revenue	o_orderdate	o_shippriority
2456423	406181.0111	1995-03-05	0
3459808	405838.6989	1995-03-04	0
492164	390324.0610	1995-02-19	0
1188320	384537.9359	1995-03-09	0
2435712	378673.0558	1995-02-26	0
4878020	378376.7952	1995-03-12	0
5521732	375153.9215	1995-03-13	0
2628192	373133.3094	1995-02-22	0
993600	371407.4595	1995-03-05	0
2300070	367371.1452	1995-03-13	0

10 rows in set
8.928 seconds

Flink SQL

Final-Q3-Top10> Rank 1	orderKey: 2456423	Revenue: 406181.01	orderDate: 1995-03-05
Final-Q3-Top10> Rank 2	orderKey: 3459808	Revenue: 405838.70	orderDate: 1995-03-04
Final-Q3-Top10> Rank 3	orderKey: 492164	Revenue: 390324.06	orderDate: 1995-02-19
Final-Q3-Top10> Rank 4	orderKey: 1188320	Revenue: 384537.94	orderDate: 1995-03-09
Final-Q3-Top10> Rank 5	orderKey: 2435712	Revenue: 378673.06	orderDate: 1995-02-26
Final-Q3-Top10> Rank 6	orderKey: 4878020	Revenue: 378376.80	orderDate: 1995-03-12
Final-Q3-Top10> Rank 7	orderKey: 5521732	Revenue: 375153.92	orderDate: 1995-03-13
Final-Q3-Top10> Rank 8	orderKey: 2628192	Revenue: 373133.31	orderDate: 1995-02-22
Final-Q3-Top10> Rank 9	orderKey: 993600	Revenue: 371407.46	orderDate: 1995-03-05
Final-Q3-Top10> Rank 10	orderKey: 2300070	Revenue: 367371.15	orderDate: 1995-03-13

5.202seconds

Flink Streaming API

Moving into the live, continuous simulation domain, the 3.2&3 approach successfully regulated data ingestion using independent rate limiters, effectively delivering the requested real-time, 5-second periodic console refresh. However, evaluating the output against the true TPC-H Q3 ground truth revealed a major data quality issue, with the Top-10 ranking accuracy

falling to approximately 70%. This degradation was caused entirely by stream-velocity mismatch. Because the Orders and Lineitem files were fed into Flink via separate, uncoordinated streams, individual child line items frequently overtook their corresponding parent orders during parallel ingestion. As a result, negative revenue deductions were applied to missing or uninitialized order states, corrupting the rolling unshippedState and introducing systemic inaccuracies into the local rankings.

Final-Q3-Top10>	Rank	1	orderKey:	5772480	Revenue:	430565.33	orderDate:	1992-08-09
Final-Q3-Top10>	Rank	2	orderKey:	1998119	Revenue:	418540.14	orderDate:	1992-08-20
Final-Q3-Top10>	Rank	3	orderKey:	2230658	Revenue:	418197.31	orderDate:	1992-08-04
Final-Q3-Top10>	Rank	4	orderKey:	1628037	Revenue:	402277.97	orderDate:	1992-08-07
Final-Q3-Top10>	Rank	5	orderKey:	1464065	Revenue:	399005.65	orderDate:	1992-08-04
Final-Q3-Top10>	Rank	6	orderKey:	3417573	Revenue:	397751.57	orderDate:	1992-08-18
Final-Q3-Top10>	Rank	7	orderKey:	2319201	Revenue:	384951.14	orderDate:	1992-08-19
Final-Q3-Top10>	Rank	8	orderKey:	356611	Revenue:	384566.87	orderDate:	1992-08-21
Final-Q3-Top10>	Rank	9	orderKey:	2465444	Revenue:	384309.27	orderDate:	1992-08-18
Final-Q3-Top10>	Rank	10	orderKey:	2334368	Revenue:	381368.02	orderDate:	1992-08-09

Benchmark

--- [TOP 10] REFRESH TIME: Mon May 25 22:38:27 CST 2026 ---									
CURRENT SIMULATED DATE: 1992-08-22									
RANK 1		OrderKey:	5772480		Unshipped Rev:	430565.33		OrderDate:	1992-08-09
RANK 2		OrderKey:	1998119		Unshipped Rev:	418540.14		OrderDate:	1992-08-20
RANK 3		OrderKey:	2230658		Unshipped Rev:	418197.31		OrderDate:	1992-08-04
RANK 4		OrderKey:	1628037		Unshipped Rev:	402277.97		OrderDate:	1992-08-07
RANK 5		OrderKey:	1464065		Unshipped Rev:	399005.65		OrderDate:	1992-08-04
RANK 6		OrderKey:	2319201		Unshipped Rev:	384951.14		OrderDate:	1992-08-19
RANK 7		OrderKey:	356611		Unshipped Rev:	384566.87		OrderDate:	1992-08-21
RANK 8		OrderKey:	2334368		Unshipped Rev:	381368.02		OrderDate:	1992-08-09
RANK 9		OrderKey:	4825255		Unshipped Rev:	381154.78		OrderDate:	1992-07-10
RANK 10		OrderKey:	5485762		Unshipped Rev:	372687.62		OrderDate:	1992-08-10

3.2 Approach

This architectural flaw was completely resolved by the final implementation, which leveraged an offline external union combined with a secondary-key pre-sorting strategy. This optimization guaranteed that every parent order was fully instantiated in Flink's local MapState microseconds before any of its child shipments arrived. Under this setup, the streaming pipeline maintained the dynamic, 5-second console refresh while achieving a flawless 100% data accuracy rate. This definitive result demonstrates that eliminating stream asynchrony at the ingestion boundary allows a stateful, parallel Flink pipeline to match the perfect correctness of a batch system while delivering ultra-low-latency, real-time analytics.

Final-Q5-Top10>	Rank	1	orderKey:	511526	Revenue:	412865.12	orderDate:	1992-06-01
Final-Q3-Top10>	Rank	2	orderKey:	2078630	Revenue:	400636.36	orderDate:	1992-06-18
Final-Q3-Top10>	Rank	3	orderKey:	2289059	Revenue:	390598.44	orderDate:	1992-06-09
Final-Q3-Top10>	Rank	4	orderKey:	2397793	Revenue:	388809.07	orderDate:	1992-05-29
Final-Q3-Top10>	Rank	5	orderKey:	5669505	Revenue:	388326.68	orderDate:	1992-06-09
Final-Q3-Top10>	Rank	6	orderKey:	94948	Revenue:	383236.53	orderDate:	1992-06-16
Final-Q3-Top10>	Rank	7	orderKey:	2884677	Revenue:	376537.92	orderDate:	1992-06-21
Final-Q3-Top10>	Rank	8	orderKey:	3748775	Revenue:	370757.83	orderDate:	1992-06-19
Final-Q3-Top10>	Rank	9	orderKey:	4479233	Revenue:	367312.79	orderDate:	1992-05-30
Final-Q3-Top10>	Rank	10	orderKey:	2334594	Revenue:	366690.45	orderDate:	1992-06-09

Benchmark

--- [TOP 10] REFRESH TIME: Mon May 25 23:21:58 CST 2026 ---									
CURRENT SIMULATED DATE: 1992-06-22									
RANK 1		OrderKey:	511526		Unshipped Rev:	412865.12		OrderDate:	1992-06-01
RANK 2		OrderKey:	2078630		Unshipped Rev:	400636.36		OrderDate:	1992-06-18
RANK 3		OrderKey:	2289059		Unshipped Rev:	390598.44		OrderDate:	1992-06-09
RANK 4		OrderKey:	2397793		Unshipped Rev:	388809.07		OrderDate:	1992-05-29
RANK 5		OrderKey:	5669505		Unshipped Rev:	388326.68		OrderDate:	1992-06-09
RANK 6		OrderKey:	94948		Unshipped Rev:	383236.53		OrderDate:	1992-06-16
RANK 7		OrderKey:	2884677		Unshipped Rev:	376537.92		OrderDate:	1992-06-21
RANK 8		OrderKey:	3748775		Unshipped Rev:	370757.83		OrderDate:	1992-06-19
RANK 9		OrderKey:	4479233		Unshipped Rev:	367312.79		OrderDate:	1992-05-30
RANK 10		OrderKey:	2334594		Unshipped Rev:	366690.45		OrderDate:	1992-06-09

3.4 Approach

5. Future Directions

While the external offline union and pre-sorting strategy achieved perfect analytical accuracy, a compelling future direction is to eliminate this external preprocessing phase entirely. Future research will focus on developing a purely online stream-alignment algorithm within Flink. By leveraging custom CoProcessFunctions, or advanced Watermark Alignment strategies, the pipeline can dynamically buffer and synchronize independent, asynchronous Orders and Lineitem streams in real-time memory.